

You're reading the documentation for an older, but still supported, version of ROS 2. For information on the latest version, please have a look at [Kilted](#).

Writing a simple publisher and subscriber (C++)

Goal: Create and run a publisher and subscriber node using C++.

Tutorial level: Beginner

Time: 20 minutes

Contents

- [Background](#)
- [Prerequisites](#)
- [Tasks](#)
 - [1 Create a package](#)
 - [2 Write the publisher node](#)
 - [3 Write the subscriber node](#)
 - [4 Build and run](#)
- [Summary](#)
- [Next steps](#)
- [Related content](#)

Background

Nodes are executable processes that communicate over the ROS graph. In this tutorial, the nodes will pass information in the form of string messages to each other over a [topic](#). The example used here is a simple “talker” and “listener” system; one node publishes data and the other subscribes to the topic so it can receive that data.

The code used in these examples can be found [here](#).

Prerequisites

In previous tutorials, you learned how to create a workspace and [create a package](#).

Tasks

1 Create a package

Open a new terminal and source your ROS 2 installation so that `ros2` commands will work.

Navigate into the `ros2_ws` directory created in a [previous tutorial](#).

Recall that packages should be created in the `src` directory, not the root of the workspace. So, navigate into `ros2_ws/src`, and run the package creation command:

```
$ ros2 pkg create --build-type ament_cmake --license Apache-2.0 cpp_pubsub
```

Your terminal will return a message verifying the creation of your package `cpp_pubsub` and all its necessary files and folders.

Navigate into `ros2_ws/src/cpp_pubsub/src`. Recall that this is the directory in any CMake package where the source files containing executables belong.

2 Write the publisher node

Download the example talker code by entering the following command:

Linux

macOS

Windows

```
$ wget -O publisher_member_function.cpp
https://raw.githubusercontent.com/ros2/examples/humble/rclcpp/topics/minimal_publisher/member_function.cpp
```

Now there will be a new file named `publisher_member_function.cpp`. Open the file using your preferred text editor.

```

#include <chrono>
#include <functional>
#include <memory>
#include <string>

#include "rclcpp/rclcpp.hpp"
#include "std_msgs/msg/string.hpp"

using namespace std::chrono_literals;

/* This example creates a subclass of Node and uses std::bind() to register a
 * member function as a callback from the timer. */

class MinimalPublisher : public rclcpp::Node
{
public:
    MinimalPublisher()
    : Node("minimal_publisher"), count_(0)
    {
        publisher_ = this->create_publisher<std_msgs::msg::String>("topic", 10);
        timer_ = this->create_wall_timer(
            500ms, std::bind(&MinimalPublisher::timer_callback, this));
    }

private:
    void timer_callback()
    {
        auto message = std_msgs::msg::String();
        message.data = "Hello, world! " + std::to_string(count_++);
        RCLCPP_INFO(this->get_logger(), "Publishing: '%s'", message.data.c_str());
        publisher_->publish(message);
    }
    rclcpp::TimerBase::SharedPtr timer_;
    rclcpp::Publisher<std_msgs::msg::String>::SharedPtr publisher_;
    size_t count_;
};

int main(int argc, char * argv[])
{
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<MinimalPublisher>());
    rclcpp::shutdown();
    return 0;
}

```

2.1 Examine the code

The top of the code includes the standard C++ headers you will be using. After the standard C++ headers is the `rclcpp/rclcpp.hpp` include which allows you to use the most common pieces of the ROS 2 system. Last is `std_msgs/msg/string.hpp`, which includes the built-in message type you will use to publish data.

```

#include <chrono>
#include <functional>
#include <memory>
#include <string>

#include "rclcpp/rclcpp.hpp"
#include "std_msgs/msg/string.hpp"

using namespace std::chrono_literals;

```

These lines represent the node's dependencies. Recall that dependencies have to be added to `package.xml` and `CMakeLists.txt`, which you'll do in the next section.

The next line creates the node class `MinimalPublisher` by inheriting from `rclcpp::Node`. Every `this` in the code is referring to the node.

```

class MinimalPublisher : public rclcpp::Node

```

The public constructor names the node `minimal_publisher` and initializes `count_` to 0. Inside the constructor, the publisher is initialized with the `String` message type, the topic name `topic`, and the required queue size to limit messages in the event of a backup. Next, `timer_` is initialized, which causes the `timer_callback` function to be executed twice a second.

```

public:
    MinimalPublisher()
    : Node("minimal_publisher"), count_(0)
    {
        publisher_ = this->create_publisher<std_msgs::msg::String>("topic", 10);
        timer_ = this->create_wall_timer(
            500ms, std::bind(&MinimalPublisher::timer_callback, this));
    }

```

The `timer_callback` function is where the message data is set and the messages are actually published. The `RCLCPP_INFO` macro ensures every published message is printed to the console.

```

private:
    void timer_callback()
    {
        auto message = std_msgs::msg::String();
        message.data = "Hello, world! " + std::to_string(count_++);
        RCLCPP_INFO(this->get_logger(), "Publishing: '%s'", message.data.c_str());
        publisher_->publish(message);
    }

```

Last is the declaration of the timer, publisher, and counter fields.

```

rclcpp::TimerBase::SharedPtr timer_;
rclcpp::Publisher<std_msgs::msg::String>::SharedPtr publisher_;
size_t count_;

```

Following the `MinimalPublisher` class is `main`, where the node actually executes. `rclcpp::init` initializes ROS 2, and `rclcpp::spin` starts processing data from the node, including callbacks from the timer.

```
int main(int argc, char * argv[])
{
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<MinimalPublisher>());
    rclcpp::shutdown();
    return 0;
}
```

2.2 Add dependencies

Navigate one level back to the `ros2_ws/src/cpp_pubsub` directory, where the `CMakeLists.txt` and `package.xml` files have been created for you.

Open `package.xml` with your text editor.

As mentioned in the [previous tutorial](#), make sure to fill in the `<description>`, `<maintainer>` and `<license>` tags:

```
<description>Examples of minimal publisher/subscriber using rclcpp</description>
<maintainer email="you@email.com">Your Name</maintainer>
<license>Apache License 2.0</license>
```

Add a new line after the `ament_cmake` buildtool dependency and paste the following dependencies corresponding to your node's include statements:

```
<depend>rclcpp</depend>
<depend>std_msgs</depend>
```

This declares the package needs `rclcpp` and `std_msgs` when its code is built and executed.

Make sure to save the file.

2.3 CMakeLists.txt

Now open the `CMakeLists.txt` file. Below the existing dependency `find_package(ament_cmake REQUIRED)`, add the lines:

```
find_package(rclcpp REQUIRED)
find_package(std_msgs REQUIRED)
```

After that, add the executable and name it `talker` so you can run your node using `ros2 run`:

```
add_executable(talker src/publisher_member_function.cpp)
ament_target_dependencies(talker rclcpp std_msgs)
```

Finally, add the `install(TARGETS...)` section so `ros2 run` can find your executable:

```
install(TARGETS
  talker
  DESTINATION lib/${PROJECT_NAME})
```

You can clean up your `CMakeLists.txt` by removing some unnecessary sections and comments, so it looks like this:

```
cmake_minimum_required(VERSION 3.5)
project/cpp_pubsub

# Default to C++14
if(NOT CMAKE_CXX_STANDARD)
    set(CMAKE_CXX_STANDARD 14)
endif()

if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
    add_compile_options(-Wall -Wextra -Wpedantic)
endif()

find_package(ament_cmake REQUIRED)
find_package(rclcpp REQUIRED)
find_package(std_msgs REQUIRED)

add_executable(talker src/publisher_member_function.cpp)
ament_target_dependencies(talker rclcpp std_msgs)

install(TARGETS
  talker
  DESTINATION lib/${PROJECT_NAME})

ament_package()
```

You could build your package now, source the local setup files, and run it, but let's create the subscriber node first so you can see the full system at work.

3 Write the subscriber node

Return to `ros2_ws/src/cpp_pubsub/src` to create the next node. Enter the following code in your terminal:

Linux macOS Windows

```
$ wget -O subscriber_member_function.cpp
https://raw.githubusercontent.com/ros2/examples/humble/rclcpp/topics/minimal_subscriber/member_function.cpp
```

Check to ensure that these files exist:

```
publisher_member_function.cpp subscriber_member_function.cpp
```

Open the `subscriber_member_function.cpp` with your text editor.

```
#include <memory>

#include "rclcpp/rclcpp.hpp"
#include "std_msgs/msg/string.hpp"
using std::placeholders::_1;

class MinimalSubscriber : public rclcpp::Node
{
public:
    MinimalSubscriber()
    : Node("minimal_subscriber")
    {
        subscription_ = this->create_subscription<std_msgs::msg::String>(
            "topic", 10, std::bind(&MinimalSubscriber::topic_callback, this, _1));
    }

private:
    void topic_callback(const std_msgs::msg::String & msg) const
    {
        RCLCPP_INFO(this->get_logger(), "I heard: '%s'", msg.data.c_str());
    }
    rclcpp::Subscription<std_msgs::msg::String>::SharedPtr subscription_;
};

int main(int argc, char * argv[])
{
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<MinimalSubscriber>());
    rclcpp::shutdown();
    return 0;
}
```

3.1 Examine the code

The subscriber node's code is nearly identical to the publisher's. Now the node is named `minimal_subscriber`, and the constructor uses the node's `create_subscription` class to execute the callback.

There is no timer because the subscriber simply responds whenever data is published to the `topic` topic.

```
public:
    MinimalSubscriber()
    : Node("minimal_subscriber")
    {
        subscription_ = this->create_subscription<std_msgs::msg::String>(
            "topic", 10, std::bind(&MinimalSubscriber::topic_callback, this, _1));
    }
```

Recall from the [topic tutorial](#) that the topic name and message type used by the publisher and subscriber must match to allow them to communicate.

The `topic_callback` function receives the string message data published over the topic, and simply writes it to the console using the `RCLCPP_INFO` macro.

The only field declaration in this class is the subscription.

```
private:
    void topic_callback(const std_msgs::msg::String & msg) const
    {
        RCLCPP_INFO(this->get_logger(), "I heard: '%s'", msg.data.c_str());
    }
    rclcpp::Subscription<std_msgs::msg::String>::SharedPtr subscription_;
```

The `main` function is exactly the same, except now it spins the `MinimalSubscriber` node. For the publisher node, spinning meant starting the timer, but for the subscriber it simply means preparing to receive messages whenever they come.

Since this node has the same dependencies as the publisher node, there's nothing new to add to `package.xml`.

3.2 CMakeLists.txt

Reopen `CMakeLists.txt` and add the executable and target for the subscriber node below the publisher's entries.

```
add_executable(listener src/subscriber_member_function.cpp)
ament_target_dependencies(listener rclcpp std_msgs)

install(TARGETS
  listener
  DESTINATION lib/${PROJECT_NAME})
```

Make sure to save the file, and then your pub/sub system should be ready.

4 Build and run

You likely already have the `rc1cpp` and `std_msgs` packages installed as part of your ROS 2 system. It's good practice to run `rosdep` in the root of your workspace (`ros2_ws`) to check for missing dependencies before building:

Linux macOS Windows

```
$ rosdep install -i --from-path src --rosdistro humble -y
```

Still in the root of your workspace, `ros2_ws` , build your new package:

Linux macOS Windows

```
$ colcon build --packages-select cpp_pubsub
```

Open a new terminal, navigate to `ros2_ws` , and source the setup files:

Linux macOS Windows

```
$ . install/setup.bash
```

Now run the talker node. The terminal should start publishing info messages every 0.5 seconds, like so:

```
$ ros2 run cpp_pubsub talker
[INFO] [minimal_publisher]: Publishing: "Hello World: 0"
[INFO] [minimal_publisher]: Publishing: "Hello World: 1"
[INFO] [minimal_publisher]: Publishing: "Hello World: 2"
[INFO] [minimal_publisher]: Publishing: "Hello World: 3"
[INFO] [minimal_publisher]: Publishing: "Hello World: 4"
```

Open another terminal, source the setup files from inside `ros2_ws` again, and then start the listener node. The listener will start printing messages to the console, starting at whatever message count the publisher is on at that time:

```
$ ros2 run cpp_pubsub listener
[INFO] [minimal_subscriber]: I heard: "Hello World: 10"
[INFO] [minimal_subscriber]: I heard: "Hello World: 11"
[INFO] [minimal_subscriber]: I heard: "Hello World: 12"
[INFO] [minimal_subscriber]: I heard: "Hello World: 13"
[INFO] [minimal_subscriber]: I heard: "Hello World: 14"
```

Enter `Ctrl+C` in each terminal to stop the nodes from spinning.

Summary

You created two nodes to publish and subscribe to data over a topic. Before compiling and running them, you added their dependencies and executables to the package configuration files.

Next steps

Next you'll create another simple ROS 2 package using the service/client model. Again, you can choose to write it in either [C++](#) or [Python](#).

Related content

There are several ways you could write a publisher and subscriber in C++; check out the `minimal_publisher` and `minimal_subscriber` packages in the [ros2/examples](#) repo.